



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING (ARTIFICIAL INTELLIGENCE & MACHINE LEARNING)

Experiment No. 3

AIM: Find minimum and maximum number from the list using Divide & Conquer methodology. Compare performance with traditional way of finding minimum and maximum.

LAB OUTCOME: 1. Analyse the time and space complexity of algorithms.
2. Apply divide and conquer approach for solving the problems.

THEORY:

Max-Min problem is to find a maximum and minimum element from the given array. We can effectively solve it using divide and conquer approach.

In the traditional approach, the maximum and minimum element can be found by comparing each element and updating Max and Min values as and when required. This approach is simple but it does $(n - 1)$ comparisons for finding max and the same number of comparisons for finding the min. It results in a total of $2(n - 1)$ comparisons. Using a divide and conquer approach, we can reduce the number of comparisons.

```
1  Algorithm StraightMaxMin(a, n, max, min)
2  // Set max to the maximum and min to the minimum of a[1 : n].
3  {
4      max := min := a[1];
5      for i := 2 to n do
6          {
7              if (a[i] > max) then max := a[i];
8              if (a[i] < min) then min := a[i];
9          }
10 }
```

Divide and conquer approach for Max. Min problem works in three stages.

- If *a*₁ is the only element in the array, *a*₁ is the maximum and minimum.
- If the array contains only two elements *a*₁ and *a*₂, then the single comparison between two elements can decide the minimum and maximum of them.
- If there are more than two elements, the algorithm divides the array from the middle and creates two subproblems. Both subproblems are treated as an independent problem and the same recursive process is applied to them. This



**DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
(ARTIFICIAL INTELLIGENCE & MACHINE LEARNING)**

division continues until subproblem size becomes one or two.

After solving two subproblems, their minimum and maximum numbers are compared to build the solution of the large problem. This process continues in a bottom-up fashion to build the solution of a parent problem.

Algorithm:

```
1  Algorithm MaxMin(i, j, max, min)
2  // a[1 : n] is a global array. Parameters i and j are integers,
3  //  $1 \leq i \leq j \leq n$ . The effect is to set max and min to the
4  // largest and smallest values in a[i : j], respectively.
5  {
6      if (i = j) then max := min := a[i]; // Small(P)
7      else if (i = j - 1) then // Another case of Small(P)
8          {
9              if (a[i] < a[j]) then
10                 {
11                     max := a[j]; min := a[i];
12                 }
13             else
14                 {
15                     max := a[i]; min := a[j];
16                 }
17         }
18     else
19         { // If P is not small, divide P into subproblems.
20           // Find where to split the set.
21           mid :=  $\lfloor (i + j) / 2 \rfloor$ ;
22           // Solve the subproblems.
23           MaxMin(i, mid, max, min);
24           MaxMin(mid + 1, j, max1, min1);
25           // Combine the solutions.
26           if (max < max1) then max := max1;
27           if (min > min1) then min := min1;
28         }
29 }
```

Time Complexities:

The conventional algorithm takes $2(n - 1)$ comparisons in worst, best and average case.



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING (ARTIFICIAL INTELLIGENCE & MACHINE LEARNING)

MAXMIN does two comparisons to determine the minimum and maximum element and creates two problems of size $n/2$, so the recurrence can be formulated as

$$T(n) = 0, \text{ if } n = 1$$

$$T(n) = 1, \text{ if } n = 2$$

$$T(n) = 2T(n/2) + 2, \text{ if } n > 2$$

Let us solve this equation using interactive approach.

$$T(n) = 2T(n/2) + 2 \dots (1)$$

By substituting n by $(n/2)$ in Equation (1)

$$T(n/2) = 2T(n/4) + 2$$

$$\Rightarrow T(n) = 2(2T(n/4) + 2) + 2$$

$$= 4T(n/4) + 4 + 2 \dots (2)$$

By substituting n by $n/4$ in Equation (1),

$$T(n/4) = 2T(n/8) + 2$$

Substitute it in Equation (1),

$$T(n) = 4[2T(n/8) + 2] + 4 + 2$$

$$= 8T(n/8) + 8 + 4 + 2$$

$$= 23 T(n/23) + 23 + 22 + 21$$

.

.

After $k - 1$ iterations



**DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
(ARTIFICIAL INTELLIGENCE & MACHINE LEARNING)**

$$\begin{aligned}\therefore T(n) &= 2^{k-1} T\left(\frac{n}{2^{k-1}}\right) + 2^{k-1} + 2^{k-2} + \dots \\ &\quad + 2^3 + 2^2 + 2^1 \\ &= 2^{k-1} T\left(\frac{n}{2^{k-1}}\right) + \sum_{i=1}^{k-1} 2^i \\ &= 2^{k-1} T\left(\frac{n}{2^{k-1}}\right) + (2^k - 2)\end{aligned}$$

$$\text{Let } n = 2^k \Rightarrow 2^{k-1} = \left(\frac{n}{2}\right)$$

$$\begin{aligned}\therefore T(n) &= \left(\frac{n}{2}\right) T\left(\frac{2^k}{2^{k-1}}\right) + (n - 2) \\ &= \left(\frac{n}{2}\right) T(2) + (n - 2)\end{aligned}$$

For $n = 2$, $T(n) = 1$ (two elements require only 1 comparison to determine min-max)

$$T(n) = \left(\frac{n}{2}\right) + (n - 2) = \left(\frac{3n}{2} - 2\right)$$

It can be observed that divide and conquer approach does only $[(3n/2) - 2]$ comparisons compared to $2(n - 1)$ comparisons of the conventional approach.

For any random pattern, this algorithm takes the same number of comparisons.

SAMPLE CODE:



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING (ARTIFICIAL INTELLIGENCE & MACHINE LEARNING)

```
#include<stdio.h>
#include<stdio.h>
int max, min;
int a[100];
void maxmin(int i, int j)
{
    int max1, min1, mid;
    if(i==j)
    {
        max = min = a[i];
    }
    else
    {
        if(i == j-1)
        {
            if(a[i] < a[j])
            {
                max = a[j];
                min = a[i];
            }
            else
            {
                max = a[i];
                min = a[j];
            }
        }
        else
        {
            mid = (i+j)/2;
            maxmin(i, mid);
            max1 = max; min1 = min;
            maxmin(mid+1, j);
            if(max < max1)
                max = max1;
            if(min > min1)
                min = min1;
        }
    }
}
```



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING (ARTIFICIAL INTELLIGENCE & MACHINE LEARNING)

```
int main ()
{
    int i, num;
    printf ("\nEnter the total number of numbers : ");
    scanf ("%d",&num);
    printf ("Enter the numbers : \n");
    for (i=1;i<=num;i++)
        scanf ("%d",&a[i]);

    max = a[0];
    min = a[0];
    maxmin(1, num);
    printf ("Minimum element in an array : %d\n", min);
    printf ("Maximum element in an array : %d\n", max);
    return 0;
}
```

OUTPUT:

```
Enter the total number of numbers : 4
Enter the numbers :
8
4
1
7
Minimum element in an array : 1
Maximum element in an array : 8
```

CONCLUSION:

This way one can find minimum and maximum from given list of numbers using divide and conquer strategy.